

CPAC Ready-Mix Concrete Delivery

AI Hackathon: Route Optimization 2026 · ทีมเทสเทสหนึ่งสองสามสี่

สรุปผลลัพธ์	ค่า
ระยะทางรวม	247 กม. (เท่ากับ exact optimum, Gap 0.00%)
จำนวนรถที่ใช้	7 จาก 8 คัน
ไซต์ที่ถึงช้าที่สุด	84 นาที (limit 90 นาที)
ผ่านกติกาก	ครบ 6 ข้อ (5 ข้อ CVRP + ความสด)
เทียบ Nearest Neighbor	สั้นกว่า 16.6% (-49 กม., -46 kg CO2 ต่อรอบ)

1. Context (บริบท / ที่มาของปัญหา)

SCG-CPAC เป็นผู้ผลิตและจำหน่ายคอนกรีตผสมเสร็จ (ready-mix concrete) รายใหญ่ในไทย ธุรกิจนี้ต่างจากสินค้าทั่วไปตรงที่คอนกรีตเริ่มก่อตัวทันทีหลังผสมเสร็จที่โรงงาน (batching plant) หากรถไม่ใช้เวลาเดินทางไปถึงไชต์นานเกินไป คอนกรีตจะแข็งตัวบางส่วนจนเทไม่ได้และต้องทิ้งถังคัน ความเสียหายเกิดทั้งต้นทุนวัสดุ ค่าเช่าของหน้างาน และคาร์บอนจากการผลิตซ้ำ

คำถามของ Case นี้: ในหนึ่งรอบการจ่ายงาน โรงงาน CPAC ต้องส่งคอนกรีตไปยังไชต์ก่อสร้าง 15 แห่ง จะจัดเส้นทางรถอย่างไรให้ส่งครบทุกไชต์ ไม่เกินความจุรถ ถึงทุกไชต์ก่อนคอนกรีตหมดอายุ และใช้ระยะทางรวมน้อยที่สุด (ประหยัดเชื้อเพลิง ลด CO2)

โจทย์นี้ต่อยอดจาก CVRP มาตรฐานที่ใช้ใน Mission 2 (A-n32-k5 / B-n31-k5 / P-n40-k5) ซึ่งทดสอบเพียงความจุรถ โดยเพิ่มมิติที่ benchmark ไม่มี คือข้อจำกัดด้านเวลา (freshness window) ซึ่งขึ้นกับลำดับการแวะของแต่ละคัน

Context Mapping: บริบทจริง → โมเดล CVRP

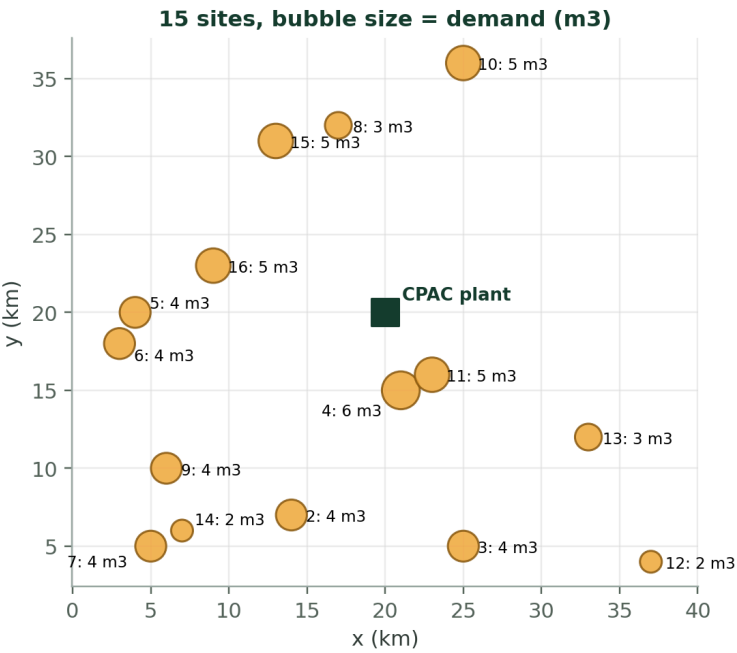
บริบทจริง	องค์ประกอบในโมเดล
CPAC Batching Plant	Depot (โหนด 1)
ไชต์ก่อสร้าง 15 แห่ง	Customers (โหนด 2-16)
ปริมาณคอนกรีตที่สั่ง (ลบ.ม.)	Demand
รถโม่ 10 ลบ.ม. จำนวน 8 คัน	Capacity + Fleet Limit
คอนกรีตต้องถึงไชต์ภายใน 90 นาที	Freshness Constraint (ใหม่)
ลดระยะทางรวม (น้ำมัน + CO2)	Objective Function

2. Data (ข้อมูล)

ไฟล์: CPAC-n16-k8-fresh.txt (รูปแบบ VRPLIB เดิม + 필드เสริม 3 ตัว: FRESHNESS_LIMIT_MIN, SPEED_KMH, SERVICE_TIME_MIN) Solver ของ Mission 2
อ่านไฟล์นี้ได้โดยไม่ต้องแก้ parser

รายการ	ค่า
Depot	CPAC Batching Plant (โหนด 1) พิกัด (20, 20)
ไชต์ก่อสร้าง	15 แห่ง (โหนด 2-16) พื้นที่ประมาณ 40 × 40 กม.
ความต้องการ	2-6 ลบ.ม. ต่อไชต์ รวม 60 ลบ.ม.
ความจุรถโม่ / จำนวนรถ	10 ลบ.ม. ต่อคัน / 8 คัน

รายการ	ค่า
ความเร็วเฉลี่ย	30 กม./ชม. (สมมติฐานการจราจรจรเขตเมือง-ปริมณฑล)
เวลาเทคอนกรีตต่อไซต์	10 นาที
Freshness limit	90 นาที นับจากรถออกจากโรงงานถึงเวลาที่ไปถึงไซต์
ระยะทาง	EUC_2D (เส้นตรง) ปิดเป็นกิโลเมตรจำนวนเต็ม



รูปที่ 1 ตำแหน่งไซต์ก่อสร้าง 15 แห่ง (ขนาดวงกลม = ปริมาณคอนกรีตที่สั่ง)

สมมติฐาน

- พิกัดและ Demand เป็นข้อมูลจำลองเพื่อการเรียนรู้ (ตามที่สโตนีโทยอนุญาต "ตัวอย่างบริบทจำลองเพื่อการเรียนรู้") ไม่ใช่ข้อมูลจริงของ SCG
- รถ 1 คันแวะส่งได้หลายไซต์ โดยสมมติเป็นงานขนาดเล็ก เช่น บ้านพักอาศัยหรืองานพื้น ในงานจริงที่เทปริมาณมาก รถมักส่งไซต์เดียวต่อเที่ยว
- นับเวลาจากรถออกจากโรงงานถึงเวลาที่ไปถึงไซต์ ส่วนนิยามที่เข้มกว่า (รวมเวลาโหลดและเวลาเทจนเสร็จ) ทดสอบไว้นหัวข้อ Sensitivity

3. Constraints (ข้อจำกัด)

รวม 6 ข้อ: 5 ข้อแรกเหมือน Mission 2 และข้อ 6 คือส่วนที่ Case นี้เพิ่มเข้ามา

- ทุกไซต์ต้องได้รับคอนกรีตครบตามที่สั่ง (Node Coverage)
- แต่ละไซต์ถูกเยี่ยมเพียงครั้งเดียว (Visited once)
- ปริมาณบรรทุกต่อคันไม่เกิน 10 ลบ.ม. (Capacity)
- ใช้รถไม่เกิน 8 คัน (Fleet Limit)
- ทุกเส้นทางเริ่มและจบที่โรงงาน (Depot Routing)
- [ใหม่] Freshness:** ทุกไซต์ต้องได้รับคอนกรีตภายใน 90 นาทีหลังรถออกจากโรงงาน เวลาถึงไซต์ที่ $k = \text{ผลรวม (ระยะทางแต่ละช่วง} \div 30 \text{ กม./ชม.)} + 10 \text{ นาที} \times \text{จำนวนไซต์ที่แวะก่อนหน้า}$

4. Method (วิธีการแก้ปัญหา)

หลักการสำคัญ: ไม่เขียน Solver ใหม่ ใช้ Solver ของ Mission 2 ตัวเดิม (อยู่ใน Route_Optimizer.ipynb ไฟล์เดียวกัน) ทุกขั้นที่สร้างหรือแก้เส้นทาง (Clarke-Wright, Relocate, Swap, 2-opt) จะถามฟังก์ชัน route_ok(route) ว่าเส้นทางใหม่ถูกกติกาหรือไม่ Mission 2 ใช้ route_ok ที่ตรวจสอบความจุ ส่วน Mission 3 ส่ง route_ok ที่ตรวจสอบทั้งความจุและเวลาเข้าไปแทน จึงไม่ต้องแก้โค้ด Solver และสอดคล้องกับหลัก Generalized Solver

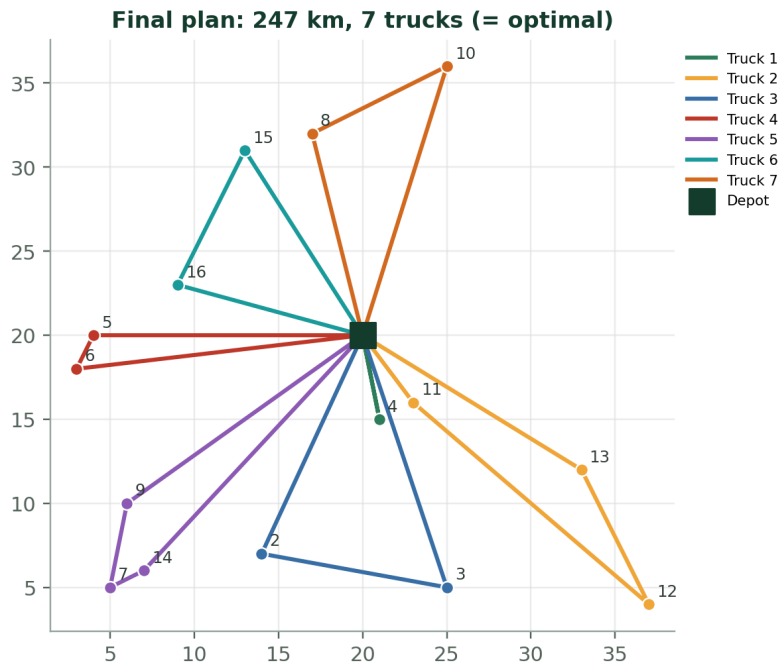
- 1. **แก้ปัญหาความจุ:** Clarke & Wright Savings สร้างเส้นทางเริ่มต้น แล้ววน Relocate / Swap / 2-opt จนไม่ดีขึ้น ได้แผนตั้งต้นที่ผ่านความจุ
- 2. **ตรวจสอบเวลาและแยกคัน:** ไล่เวลาสะสมของแต่ละคัน ถ้าไซต์ถัดไปจะถึงช้ากว่า 90 นาที ให้ตัดเส้นทางตรงนั้นแล้วให้รถคันใหม่ออกจากโรงงานไปส่งไซต์ที่เหลือ ทำให้ทุกไซต์ทันเวลา
- 3. **ปรับปรุงแบบรู้เวลา:** รัน Relocate / Swap / 2-opt ชุดเดิมอีกรอบ โดยทุกการเปลี่ยนแปลงต้องผ่านทั้งความจุและเวลา Relocate ย้ายลูกค้าออกจากรถที่ไม่มีจุดเดียวได้ และถ้ารถว่างก็ยุบคันนั้น คันนี้รวมไซต์ที่อยู่ทิศเดียวกันกลับเข้าคันเดียว เช่น 11 → 12 → 13 จึงไม่ใช้รถเกินจำเป็น
- 4. **ตรวจสอบและวัดคุณภาพ:** ตรวจสอบครบ 6 กติกา แล้วเทียบกับ (n) exact optimum ที่คำนวณด้วย set partitioning (ทำได้เพราะเคสมี 15 ไซต์) และ (ข) baseline Nearest Neighbor ที่ทำตามกติกาเดียวกัน ทุกตัวเลขรันได้ใน Route_Optimizer.ipynb

5. Result & Insight (ผลลัพธ์และสิ่งที่ได้เรียนรู้)

Result Card

คัน	เส้นทาง	ลบ.ม.	กม.	ถึงช้าสุด (นาที)
1	Plant → 4 → Plant	6	10	10
2	Plant → 11 → 12 → 13 → Plant	10	47	84
3	Plant → 2 → 3 → Plant	8	41	60
4	Plant → 5 → 6 → Plant	8	35	46
5	Plant → 14 → 7 → 9 → Plant	10	43	72
6	Plant → 16 → 15 → Plant	10	33	50
7	Plant → 10 → 8 → Plant	8	38	62
รวม	7 คัน	60	247	สูงสุด 84

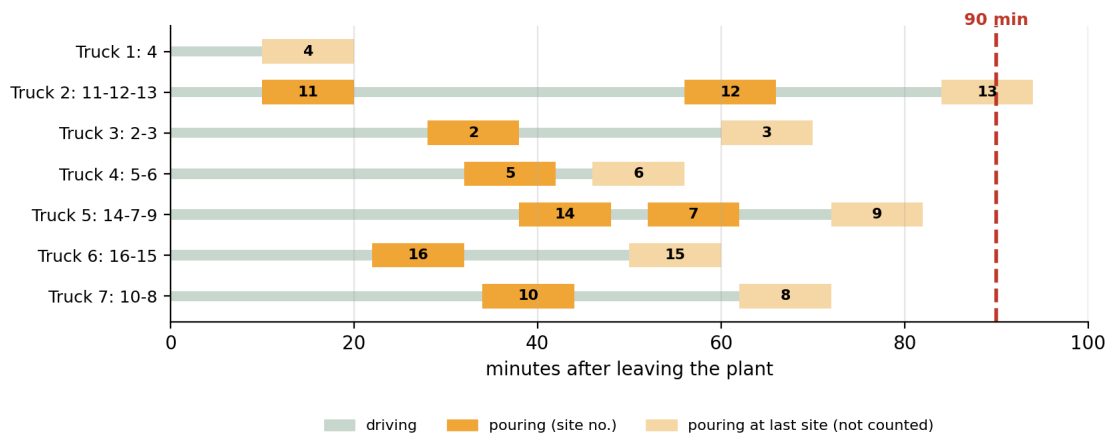
ผลการตรวจ: Node Coverage ผ่าน · Visited once ผ่าน · Capacity ผ่าน (ทุกคัน ≤ 10 ลบ.ม.) · Fleet Limit ผ่าน (7/8) · Depot Routing ผ่าน · Freshness ผ่าน
ทุกคัน (≤ 84 นาที)



รูปที่ 2 แผนเส้นทางสุดท้าย 247 กม. 7 คัน

ไทม์ไลน์ของรถแต่ละคัน

นาฬิกาเริ่มนับเมื่อรถออกจากโรงงาน ขับ 1 กม. ใช้เวลา 2 นาที (30 กม./ชม.) เวลาเท 10 นาทีที่ไซต์ก่อนหน้าทำให้ไซต์ถัดไปถึงช้าลง กติกาคือเวลาที่ไปถึงทุกไซต์ต้องไม่เกิน 90 นาที ส่วนเวลาที่ไซต์สุดท้ายและขากลับโรงงานไม่นับ (ไม่ว่างแล้ว)



รูปที่ 3 ไทม์ไลน์ของรถ 7 คัน (แถบเทาอ่อน = ขับรถ, แถบส้ม = เทคอนกรีต)

คัน	ไทม์ไลน์ (นาทีหลังจากออกจากโรงงาน)	ถึงไซต์สุดท้าย	เทเสร็จ
1	ถึง 4 นาทีที่ 10	10	20
2	ถึง 11 นาทีที่ 10 → ถึง 12 นาทีที่ 56 → ถึง 13 นาทีที่ 84	84	94
3	ถึง 2 นาทีที่ 28 → ถึง 3 นาทีที่ 60	60	70
4	ถึง 5 นาทีที่ 32 → ถึง 6 นาทีที่ 46	46	56
5	ถึง 14 นาทีที่ 38 → ถึง 7 นาทีที่ 52 → ถึง 9 นาทีที่ 72	72	82
6	ถึง 16 นาทีที่ 22 → ถึง 15 นาทีที่ 50	50	60
7	ถึง 10 นาทีที่ 34 → ถึง 8 นาทีที่ 62	62	72

ทำไมลำดับการแวะสำคัญ

ไฮด์ 12 และ 13 อยู่ไกลทางตะวันออกเฉียงใต้ของโรงงาน แต่ละแบบด้านล่างเปลี่ยนเพียงไฮด์ที่แวะก่อนหรือลำดับการแวะ

เส้นทาง	ไทม์ไลน์	ผล
Plant → 3 → 12 → 13	ถึง 3 นาที่ที่ 32 → ถึง 12 นาที่ที่ 66 → ถึง 13 นาที่ที่ 94	ไม่ผ่าน (สาย 4 นาที่)
Plant → 11 → 12 → 13	ถึง 11 นาที่ที่ 10 → ถึง 12 นาที่ที่ 56 → ถึง 13 นาที่ที่ 84	ผ่าน (แผนที่เลือก)
Plant → 13 → 12 → 11	ไฮด์เดียวกัน แต่ไปไกลก่อน ถึง 11 นาที่ที่ 104	ไม่ผ่าน
Plant → 12 → 13 → 11	ถึง 11 นาที่ที่ 106	ไม่ผ่าน

- ไฮด์ชุดเดียวกันอาจผ่านหรือไม่ผ่านก็ได้ ขึ้นกับลำดับ Solver จึงต้องตรวจเวลาทุกครั้งที่ย้ายหรือสลับลูกค้า
- เวลาเหมาะสมตามจำนวนไฮด์ รถที่แวะ 3 ไฮด์เสียเวลา 20 นาทีก่อนถึงไฮด์สุดท้าย เหลือเวลาขับประมาณ 70 นาที (≈35 กม.)
- ไฮด์ที่อยู่ใกล้ควรแวะก่อน ไฮด์ที่อยู่ไกลควรอยู่ท้ายเส้นทาง

เปรียบเทียบกับแผนอ้างอิง

แผน	กม.	คัน	ใช้งานได้จริง
ดูแค่ความจุ (ไม่มีกติกาลเวลา)	241	7	ไม่ได้ (1 เส้นทางถึงนาที่ที่ 94)
Solver ของทีม (รวมกติกาลเวลา)	247	7	ได้
Exact optimum (อ้างอิง)	247	7	ได้
Nearest Neighbor (baseline)	296	8	ได้

Insight ที่สำคัญที่สุด

- 1) ต้นทุนของกติกาลความสดเล็กน้อยที่คิด: แผนที่ดูแค่ความจุได้ 241 กม. แต่ใช้จริงไม่ได้ แผนที่ผ่านกติกาลเวลาและดีที่สุดคือ 247 กม. หมายความว่าความสดของคอนกรีตแลกมาด้วยระยะเพิ่มเพียง 6 กม. (+2.5%) และไม่ต้องเพิ่มรถ
- 2) ลำดับการแวะสำคัญพอๆ กับการจัดกลุ่ม: ไฮด์ชุดเดียวกันอาจทันหรือไม่ทันเวลาก็ได้ ขึ้นกับว่าแวะไฮด์ไหนก่อน Solver ที่ตรวจเวลาทุกครั้งที่ย้ายหรือสลับลูกค้าจึงหาแผนที่ทั้งทันเวลาและสั้นที่สุดได้ ได้ 247 กม. เท่ากับ exact optimum และสั้นกว่า Nearest Neighbor 16.6%
- 3) Generalized Solver ใช้ได้จริง: กติกาใหม่ถูกเพิ่มผ่านฟังก์ชัน route_ok เพียงจุดเดียว ไม่ต้องแก้ solver ของ Mission 2 และผลลัพธ์ของ Mission 2 ไม่ได้รับผลกระทบ

เชื่อมโยงกับ Green Logistics

คำนวณ CO2 = ระยะทาง (กม.) × 0.35 ลิตร/กม. × 2.68 kg CO2/ลิตรดีเซล (อัตราสิ้นเปลืองเป็นสมมติฐานของรถไม่ขังบรรทุกเต็ม ควรยืนยันกับข้อมูล fleet จริง ส่วน 2.68 อิงค่า default ของ IPCC)

แผน	กม.	CO2 ต่อรอบ (kg)
Nearest Neighbor	296	277.6
Solver ของทีม (= optimal)	247	231.7

- เทียบ Nearest Neighbor ลดได้ 49 กม. หรือประมาณ 46 kg CO2 ต่อรอบ ถ้าจ่ายงานวันละ 1 รอบ 300 วันต่อปี เท่ากับประมาณ 13.8 ตัน CO2 ต่อปี (ตัวอย่างเพื่อแสดงขนาดผลกระทบ)

- กติกาความสดเพิ่มระยะเพียง 6 กม. (ประมาณ 5.6 kg CO2) แต่ป้องกันคอนกรีตเสียได้ทั้งเที่ยว คาร์บอนจากปูนซีเมนต์ในคอนกรีตอยู่ในระดับหลักร้อย kg ต่อ ลบ.ม. จึงสูงกว่าหลายเท่า (ควรอ้างค่าจาก EPD ของ CPAC เมื่อนำเสนอจริง)

Sensitivity: ถ้านิยาม 90 นาทีเพิ่มขึ้น

ถ้ากำหนดให้ต้องเสร็จภายใน 90 นาที เวลาไปถึงไซต์ต้องไม่เกิน 80 นาที และถ้านับเวลาโหลดที่โรงงานอีก 10 นาทีด้วย จะเหลือ 70 นาที ผลเมื่อปรับ limit เป็นดังนี้

เวลาถึงไซต์สูงสุด	Solver (กม. / คัน)	Exact (กม. / คัน)	Gap
90 นาที (Case หลัก)	247 / 7	247 / 7	0.00%
80 นาที (เสร็จใน 90)	248 / 7	248 / 7	0.00%
70 นาที (รวมเวลาโหลด)	258 / 7	248 / 7	4.03%
60 นาที	289 / 8	277 / 8	4.33%

ถ้าต้องเสร็จภายใน 90 นาที แผนเปลี่ยนเป็น 248 กม. 7 คัน ซึ่งยังเท่ากับ optimal (เพิ่มแค่ 1 กม.) และแม้นิยามเข้มถึง 70 นาที ก็ยังใช้รถ 7 คัน แต่เมื่อเวลาถึงมาก local search แบบย้ายทีละจุดเริ่มห่างจาก optimal ประมาณ 4% ซึ่งชี้ทิศทางการพัฒนาต่อ

6. ข้อจำกัดและแนวทางพัฒนาต่อ

ข้อจำกัด	แนวทางพัฒนาต่อ
ติด local optimum (P-n40-k5 Gap 12.23%)	Multi-start / Simulated Annealing / Tabu Search
ย้ายหรือสลับทีละ 1 จุด (Gap ~4% เมื่อเวลาถึง)	Or-opt ย้ายเป็นกลุ่ม 2-3 จุดติดกัน
มีเส้นไขว้ระหว่างคัน (2-opt ทำงานในคันเดียว)	2-opt* ให้ 2 คันแลกช่วงเส้นทางกัน
ระยะเส้นตรง ความเร็วคงที่ ระบายงานเดียว	ระยะถนนจริง + traffic + รถวิ่งหลายเที่ยว (Google OR-Tools)
ไม่มี time window ของไซต์ และไม่แบ่งส่ง	VRPTW + split delivery สำหรับงานขนาดใหญ่

7. ไฟล์ที่เกี่ยวข้อง

- **Route_Optimizer.ipynb** — Solver ทั้ง Mission 2 และ Mission 3 ในไฟล์เดียว กด Run all บน Google Colab ได้ทันที มี Result Card กราฟ การเปรียบเทียบ CO2 และ sensitivity
- **CPAC-n16-k8-fresh.txt** — ข้อมูลของ Case นี้ในรูปแบบ VRPLIB (ข้อมูลเดียวกันฝังไว้ใน notebook ด้วย)